

面向污水处理与环境监测的

AutoIF 环境工程大模型

指令遵循优化系统

完整操作指南

从 AutoDL 租机、环境安装、一键训练，到评测、排错与关机

主线：污水处理 第二场景：环境监测 扩展：废气 / 固废 / 噪声 / 环评

请勿沿用旧材料中互相矛盾的 37/406 条 SFT 或两套 DPO 学习率

完整版（从租机到评测、排错、关机）

项目全称：面向污水处理与环境监测的 AutoIF 环境工程大模型指令遵循优化系统

适用对象：需要在 AutoDL 上复现本项目的同学；需要把操作写进报告/答辩材料的同学。

配套代码目录：AutoIF-LLM/

实验数字唯一来源：output/experiment_summary.json（未重跑完成前，训练条数、Loss、量化精度必须保持空白或写「待实验填写」，禁止抄旧材料里的 37 条或 406 条。）

0. 先读这一页（否则后面容易做错）

0.1 这个项目做什么

本项目不是把通用问答模型换成环保词汇，而是：

1. 用可程序验证的环境工程种子指令驱动 AutoIF；
2. 用领域查询集（污水处理为主、环境监测为辅）而不是生活/编程/法律问题；
3. 用「格式 + 计算 + 专业依据」三层验证生成 SFT / DPO 数据；
4. 用 LoRA 微调 Qwen2.5-1.5B，再用 DPO 拉开「算对、单位对、不编造标准」和各类错误回答的差距。

目标是优化结构化专业回答能力，不是宣称少量微调就能掌握全部环境工程知识。

0.2 这个项目不做什么

- 不宣称训练出一个「环境工程专家大模型」。
- 不把 Loss 下降当成果证明。
- 不预先宣称「量化只掉 2%~3%」。
- 不在未提供标准名称、适用对象、年份时让模型编造排放限值。
- 不要再用「法律 / 医疗 / 金融」当主演示路径。

0.3 推荐主路径（请按这个跑）

```
cd /root/autodl-tmp/AutoIF-LLM
bash setup.sh
nohup bash run_all.sh --domain 环境工程 \
--query-file data/environmental_engineering_queries.jsonl \
--test-file data/environmental_engineering_test.jsonl \
> run.log 2>&1 &
tail -f run.log
```

默认领域就是环境工程。污水处理、环境监测 是别名，共用同一套种子指令。

0.4 时间与费用（按量计费粗估）

阶段	大约耗时	说明
租机 + 上传	5~10 分钟	选错镜像会后面全废
setup.sh	约 30 分钟	含模型下载，视网速
run_all.sh	约 20 分钟	不含安装；数据合成占大部分
四组评测	视测试条数	正式评测建议跑完整测试集
合计	约 1 小时	A800 约 2~4 元

充值 10 元通常够跑完全流程。跑完必须在 AutoDL 网页关机，否则按量计费会一直扣。

0.5 统一超参（不要写两套学习率）

来源：configs/pipeline_config.yaml, run_all.sh 会读取。

项目	SFT	DPO
学习率	5e-5	5e-6
轮数	3	2
LoRA rank / alpha	8 / 16	8 / 16
batch size	4	2
gradient accumulation	4	4
序列长度	2048	2048
pref_beta	—	0.1

1. 电脑和账号准备

1.1 你需要有的东西

- AutoDL 账号：<https://www.autodl.com>
- 本项目文件夹：本地的 AutoIF-LLM/（或打成 zip 后上传）
- 能打开 JupyterLab 的浏览器
- 可选：本地终端（Windows 可用 PowerShell），以便用 scp 传大模型

1.2 建议的 GPU 与镜像（必须选对）

1. 打开 AutoDL → 容器实例 → 创建新实例。
2. 地区：华北或华东（延迟低即可，不强制）。
3. GPU：A800-80GB。若无，选 A100-80GB 或 A800-40GB。40GB 卡训练时可能要减小 batch size。
4. 镜像（最关键）：
 - 标签选「基础镜像」或「社区镜像」；
 - 搜索 PyTorch；
 - 选 PyTorch 2.4.0 + Python 3.12 + CUDA 12.1；
 - 没有 2.4.0 时，PyTorch 2.3.x + CUDA 12.1 也可，setup.sh 会再装 2.4.0。
5. 计费：按量计费。
6. 数据盘：免费赠送即可，不必另买。
7. 创建后等 1~2 分钟，点 JupyterLab 进入。

选错镜像的后果：

- CUDA 11.x → vLLM 装不上；
- PyTorch 2.5+ → 与 vLLM 0.5.5 不兼容。

1.3 路径铁律

项目和模型必须放在数据盘：

```
/root/autodl-tmp/AutoIF-LLM
```

不要放在 /root/。系统盘大约 30GB，教师模型约 15GB，学生模型约 3GB，NLI 约 2.5GB，放系统盘会报 No space left on device。

2. 上传与解压

2.1 方法一：JupyterLab 拖拽（推荐）

- 1. 左侧文件树进入 autodl-tmp。
- 2. 把整个 AutoIF-LLM 文件夹打成 zip，或直接拖文件夹（若界面支持）。
- 3. 若上传的是 zip：

```
cd /root/autodl-tmp
unzip AutoIF-LLM.zip
cd AutoIF-LLM
pwd # 确认当前就在项目根目录
ls # 应能看到 setup.sh、run_all.sh、code、data、configs
```

如果 zip 解压后多了一层 AutoIF-LLM/AutoIF-LLM，进入内层那个含 setup.sh 的目录。

2.2 方法二：scp

在 AutoDL 实例详情页看 SSH 地址和端口。在本地电脑执行（替换端口和主机名）：

```
scp -P 端口号 AutoIF-LLM.zip root@connect.xxx.sectacloud.com:/root/autodl-tmp/
```

然后在 JupyterLab 终端里 unzip。

2.3 上传后最少应看到的文件

路径	作用
setup.sh	装环境、下模型
run_all.sh	一键全流程
configs/pipeline_config.yaml	阈值和学习率唯一来源
sample_data/seed_instruction_环境工程.txt	40 条可验证种子
data/environmental_engineering_queries.jsonl	领域查询（当前 412 条）
data/environmental_engineering_test.jsonl	独立测试集（当前 173 条，与训练集查询无重叠）
code/	AutoIF 九步
code_dpo/	DPO 三步
scripts/	评测、汇总、演示

当前数据规模以你机器上的 output/experiment_summary.json 里 counts 为准。上面 412 / 173 / 40 是生成器当前产物，若你重新跑了 python scripts/generate_env_queries.py，以新文件为准。

3. 一键安装环境（setup.sh）

在项目根目录：

```
bash setup.sh
```

大约 30 分钟。它固定做 5 件事：

步	做什么	成功时你应看到
1/5	清华镜像安装 PyTorch 2.4.0+cu121、transformers、vLLM 0.5.5，并修补 pyairports	依赖安装完成
2/5	克隆并安装 LlamaFactory（先 gitee，失败再 github）	LlamaFactory 安装完成
3/5	ModelScope 下 Qwen2.5-7B-Instruct、Qwen2.5-1.5B；HF 镜像下 mDeBERTa NLI	模型下载完成
4/5	解析 modelscope 深层目录，尝试注册数据集	打印教师/学生真实路径
5/5	打印 PyTorch / CUDA / GPU / vLLM / LlamaFactory	环境验证完成

结束标志： 终端出现 环境配置完成。

3.1 建议再手跑一遍自检

```
python -c "import torch; print('PyTorch:', torch.__version__, '| CUDA:', torch.cuda.is_available())"
python -c "import vllm; print('vLLM:', vllm.__version__)"
python -c "import llamafactory; print('LlamaFactory: OK')"
ls models/teacher models/student models/nli
nvidia-smi
```

四条 Python/目录命令不报错，且 nvidia-smi 能看到 GPU，再进入 run_all.sh。

3.2 安装失败怎么处理

磁盘满：

```
df -h
# 若项目在 /root，挪到数据盘
mv /root/AutoIF-LLM /root/autodl-tmp/
cd /root/autodl-tmp/AutoIF-LLM
bash setup.sh
```

下载卡住： 直接再执行一次 bash setup.sh。脚本对已存在的 models/teacher、models/student、models/nli 会跳过。

vLLM 导入失败： 确认镜像是 CUDA 12.1，不要混装其它 torch。setup.sh 会强制装 2.4.0。

LlamaFactory 克隆失败： 在项目根手动：

```
git clone --depth 1 https://gitee.com/hiyouga/LLaMA-Factory.git LlamaFactory
cd LlamaFactory
pip install -e "[torch,metrics]" -i https://pypi.tuna.tsinghua.edu.cn/simple
cd ..
```

4. 一键运行全流程（run_all.sh）

4.1 推荐命令（后台，可关浏览器）

```
cd /root/autodl-tmp/AutoIF-LLM
nohup bash run_all.sh --domain 环境工程 \
--query-file data/environmental_engineering_queries.jsonl \
--test-file data/environmental_engineering_test.jsonl \
> run.log 2>&1 &
echo $! # 记下进程号
tail -f run.log
```

看到 全流程完成 即结束。用 Ctrl+C 只是退出 tail，不会停训练（因为用了 nohup）。

千万不要在前台跑 `bash run_all.sh` 然后关掉 JupyterLab 标签页——进程会被杀，需要重跑。

4.2 全部命令行参数

参数	默认	含义
<code>--domain</code>	环境工程	选用 <code>sample_data/seed_instruction_<领域>.txt</code>
<code>--query-file</code>	YAML 里的 <code>data.query_file</code>	领域查询 JSONL
<code>--test-file</code>	YAML 里的 <code>evaluation.test_set</code>	冒烟测试 / 量化校准语料
<code>--skip-train</code>	关	只做数据合成 + 汇总，不训练

只合成数据、暂不训练：

```
bash run_all.sh --domain 环境工程 --skip-train
```

4.3 脚本会自动做、你不必手做的事

- 从 YAML 读取 SFT/DPO 学习率、batch、epoch；
- 设置 `AUTOIF_SEED_PATH`、`AUTOIF_QUERY_PATH`、`AUTOIF_DOMAIN`；
- 使用 `sample_data/seed_instruction_环境工程.txt`；
- `trap` 在退出时关闭 vLLM，避免占着显存；
- SFT 数据用 `scripts/prepare_llamafactory_data.py` 复制 `output/IF_sft_data.json`（字段是 `output`，不是 `gpt-answer`）；
- DPO 保留 ShareGPT 的 `conversations / chosen / rejected`。

4.4 内部 8 个阶段（对应日志）

阶段	内容	日志
1	启动 vLLM 教师模型 Qwen2.5-7B-Instruct，端口 8000， <code>gpu-memory-utilization 0.5</code>	终端
2	AutoIF 九步 + DPO-1/2/3	<code>logs/step1.log ... step9.log</code> ， <code>logs/dpo1.log</code> <code>dpo2.log dpo3.log</code>
3	关闭 vLLM，把显存留给训练	—
4	LoRA SFT	<code>logs/sft_train.log</code>
5	LoRA DPO（学习率 $5e-6$ ）	<code>logs/dpo_train.log</code>
6	合并 LoRA 到学生模型	<code>logs/merge.log</code>
7	GPTQ 量化（环境工程校准语料；失败可忽略）	<code>logs/quantize.log</code>
8	用测试集前 3 条冒烟推理，并生成 <code>output/experiment_summary.json</code>	<code>logs/inference_test.log</code>

阶段 7 失败是常见现象，不影响 `models/model_merged/`。不要把量化失败写成项目失败。

4.5 教师 / 学生 / NLI 角色

角色	模型	大约体积	用途
教师	Qwen2.5-7B-Instruct	15GB	指令增强、写验证函数、生成回答、打质量分
学生	Qwen2.5-1.5B	3GB	被 LoRA 微调的目标模型
NLI	mDeBERTa-v3	2.5GB	步骤 5：指令与验证函数是否说同一件事

环境变量（run_all.sh 已 export，逐步调试时要自己 export）：

```
export SUPERVISOR_API_BASE="http://localhost:8000/v1"
export SUPERVISOR_API_KEY="EMPTY"
export SUPERVISOR_MODEL="Qwen/Qwen2.5-7B-Instruct"
export NLI_MODEL_PATH="/models/nli"
export HF_ENDPOINT="https://hf-mirror.com"
export AUTOIF_DOMAIN="环境工程"
export AUTOIF_SEED_PATH="sample_data/seed_instruction_环境工程.txt"
export AUTOIF_QUERY_PATH="data/environmental_engineering_queries.jsonl"
export AUTOIF_TEST_PATH="data/environmental_engineering_test.jsonl"
```

SUPERVISOR_MODEL 必须与 vLLM 的 --served-model-name 逐字相同，否则步骤 1 起会 404。

5. AutoIF 九步在环境工程里具体做什么

每一步都在项目根目录执行。输入输出文件均在 output/（种子和查询在 sample_data/、data/）。

Step 1 指令增强 `python code/1_RFT.py`

- 输入：AUTOIF_SEED_PATH（默认 40 条环境工程约束）。
- 输出：output/augment_instructions.txt。
- 教师模型按种子再写约几十到一百多条「可用 Python 检查」的格式指令。
- 提示词已限制：必须适用于污水处理/监测等，禁止法律医疗编程金融和生活闲聊，禁止写无法抽数字/标题/表格的文风指令。

Step 2 验证函数生成 `python code/2_verification_funcs_cases_generation.py`

- 输入：种子 + 增强指令。
- 输出：output/eval_func_rft.jsonl。
- 为每条指令生成 Python evaluate(response) 和 3 个测试用例。
- 额外把 code/env_seed_validators.py 里的确定性函数注入种子指令（去除率字段、单位白名单、「信息不足」前缀等），避免只检查有没有出现「COD」三个字。

Step 3 交叉验证 `python code/3_cross_validation.py`

- 输出：output/cross_validation.jsonl。
- 用测试用例筛函数，用函数筛用例，准确率阈值默认 0.8（YAML：data.quality_thresholds.cross_validation_acc）。
- 种子确定性函数会保底进入候选。

Step 4 反向翻译 `python code/4_eval_func_backtranslator.py`

- 把验证函数再翻译回自然语言，供下一步 NLI 对齐。
- 此文件保持原 AutoIF 逻辑，一般不用改。

Step 5 NLI 过滤 `python code/5_eval_func_backtranslator_filter.py`

- 用 NLI 模型判断「指令」和「验证函数描述」是否一致。
- 输出：output/back_trans_filter.jsonl。
- 此文件保持原逻辑。

Step 6 领域查询配对 + 生成回答 `python code/6_concat_sharegpt_query.py`

这是专业化最关键的一步。

- 优先读 `data/environmental_engineering_queries.jsonl`（保留 `category`、`gold`）；
- 没有足够查询时，才用 LLM 按环境工程约束生成，不再生成生活常识题；
- 每条指令默认配 16 条查询（YAML: `data.query.queries_per_instruction`）；
- 每条配对让教师模型生成 $K=5$ 个回答；
- `prompt` 里同时放格式要求和环境问题，并写明：未给标准不得编造限值。

输出: `output/query_rft.jsonl`。

Step 7 三层验证 + 教师质量分 `python code/7_query_verification.py`

- 先跑 AutoIF 的格式验证函数；
- 再跑 `code/env_validators.py`: 格式 / 计算 / 依据，写入 `layer_score`；
- 计算层失败不会直接丢样本（教师模型也可能算错），留给 DPO 做偏好；
- 再让教师模型打 0~10 分，末行必须是 `Score: x`。

输出: `output/query_rft_score.jsonl`。

Step 8 质量过滤 `python code/8_query_score_filter.py`

- 默认保留平均分 > 8 （YAML: `data.quality_thresholds.quality_score`）。
- 数据太少时把该值改成 5 或 6，只重跑 8、9 步。
- 输出: `output/query_score_filter.jsonl`，答案字段是 `response`。

Step 9 构建 SFT `python code/9_sft_data_construction.py`

- 把 `query + instruction` 拼成一条用户指令，`response` 作为 `output`。
- 若 `response` 为空，才回退读 `gpt-answer`。
- 输出: `output/IF_sft_data.json`（Alpaca: `instruction / input / output`）。
- 训练必须用这个文件，不要让 `shell` 再去读 `gpt-answer`。

SFT 回答结构建议（种子和生成提示已朝这个方向约束）：

1. 已知条件；
2. 公式或判断依据；
3. 计算/分析过程；
4. 结论；
5. 假设与风险。

6. DPO 三步

DPO-1 `python code_dpo/1_dpo_rft_wash.py`

对步骤 6 的多个回答用验证函数打 0~1 分，并保留完整 `query` 和 `instruction`，不再把「整段 `prompt`」误当成唯一字段。

DPO-2 `python code_dpo/2_dpo_data_query_construct.py`

- `human` 侧为：环境问题 + 换行 + 【格式要求】+ 指令，查询不会丢失。
- `chosen / rejected` 写成 ShareGPT 对象: `{"from": "gpt", "value": "..."}。`
- 若样本带 `gold`，计算错误的回答不得进入 `chosen`。

DPO-3 `python code_dpo/3_env_hard_negatives.py`

有计划地追加硬负样本，例如：

- 公式对、单位错（`kg/d` 写成 `t/d` 或 `ppm`）；
- 数值算错；
- 只给结论；

- 编造 GB/HJ 限值；
- 缺数据却给出确定结论。

chosen 不能只是更长的回答，而应是计算正确、依据透明、边界清楚。

7. 训练、合并、量化、冒烟测试

run_all.sh 在数据合成后会：

```
python scripts/prepare_llamafactory_data.py
```

它会：

- 复制 output/IF_sft_data.json → LlamaFactory/data/autoif_sft.json；
- 把 output/dpo_pairs.jsonl 规范成 ShareGPT 写入 autoif_dpo.json；
- 注册 dataset_info.json。

然后在 LlamaFactory 目录调用 llamafactory-cli train。学习率以 YAML 为准：SFT 5e-5，DPO 5e-6。

合并：

```
学生模型 + models/model_c_dpo → models/model_merged
```

量化：scripts/quantize_env.py 用测试集题干做校准。失败则忽略。量化前后专业指标必须实测，禁止事先写「只掉 2%~3%」。

冒烟：scripts/smoke_infer.py 抽测试集前 3 条。这不是正式评测。

最后：

```
python scripts/build_experiment_summary.py
```

生成 output/experiment_summary.json。报告、PPT、PDF 里的条数、过滤率、Loss 只许抄这一份。

8. 跑完后看什么

8.1 必看汇总

```
python scripts/build_experiment_summary.py
cat output/experiment_summary.json
```

关注 counts.sft、counts.dpo_pairs、各步过滤后的行数、training.sft_last_loss。若 sft 仍是 0，说明训练没完成或数据没写成 IF_sft_data.json。

8.2 手算条数

```
python -c "import json; d=json.load(open('output/IF_sft_data.json',encoding='utf-8')); print('SFT', len(d), '空答案', sum(1 for x in d if not str(x.get('output','')).strip()))"
wc -l output/dpo_pairs.jsonl
tail -20 logs/sft_train.log
tail -20 logs/dpo_train.log
cat logs/inference_test.log
du -sh models/model_c_sft models/model_c_dpo models/model_merged
```

空答案应当接近 0。若很多空答案，说明又走了错误字段，检查是否用了 prepare_llamafactory_data.py。

8.3 产物对照表

路径	含义
output/IF_sft_data.json	SFT 真源

路径	含义
output/dpo_pairs.jsonl	DPO 真源（ShareGPT）
output/query_score_filter.jsonl	过滤后原始样本（字段 response）
output/experiment_summary.json	文档唯一数字源
LlamaFactory/data/autoif_sft.json	训练用副本
models/model_c_sft/	SFT LoRA
models/model_c_dpo/	DPO LoRA
models/model_merged/	可独立加载的最终模型
models/model_gptq_int4/	量化模型（可能没有）
logs/	每步日志
eval_results/	正式评测输出

9. 正式评测（四组对比）

不要只用冒烟问答。至少比较：

- 1. 基座学生模型 models/student（实际路径以 config.json 所在目录为准）
- 2. SFT LoRA（需按你们 LlamaFactory 加载方式，或只评合并前的生成脚本）
- 3. SFT+DPO 合并模型 models/model_merged
- 4. 量化模型 models/model_gptq_int4（若存在）

```
# 用已有预测文件（每行 query + response）
python scripts/eval_env_engineering.py \
--pred-file eval_results/某个模型_preds.jsonl \
--tag sft_dpo

# 直接加载合并模型在测试集上生成（需要 GPU）
python scripts/eval_env_engineering.py \
--model-path ./models/model_merged \
--tag sft_dpo
```

指标含义：

指标	含义
指令遵循准确率	格式约束通过比例
计算正确率	抽出的数字与 gold 在容差内（默认相对误差 2%）
单位正确率	如负荷是否为 kg/d
字段完整率	规定章节/标题是否齐全
虚构标准率	题面未给标准却写出 GB/限值且未标注需核实
不确定性处理率	缺参时是否以「信息不足」等方式拒绝确定结论
量化性能损失	同一测试集上量化前后上述指标差
延迟与显存	部署效率，与专业正确率分开报

本地无 GPU 时可先演示验证器：

```
python tests/test_env_validators.py
python scripts/demo_env_autoif.py
python scripts/eval_env_engineering.py --pred-file eval_results/demo_preds.jsonl --tag demo --limit 12
```

COD 标准例题（介绍项目时建议现场算）：

- 水量 10000 m³/d，进水 COD 300 mg/L，出水 50 mg/L；
- 去除率 83.33%；
- 日去除负荷 2500 kg/d。

正确过程应通过三层验证：只给结论、算成 50%、写成 t/d、编造 GB 18918 限值，都应被打回。

10. 配置怎么改（不要改 `utils.py` 里的魔数）

只改 `configs/pipeline_config.yaml`，然后重跑对应步骤。

常用项：

YAML 路径	默认	何时改
<code>data.quality_thresholds.quality_score</code>	8	SFT 太少，降到 5~6 后重跑 step8+9
<code>data.query.queries_per_instruction</code>	16	想增加配对量（更慢、更贵）
<code>data.validation.k_response</code>	5	每个问题生成几个回答
<code>training.sft.batch_size</code>	4	40GB 卡 OOM 时改为 2 或 1
<code>training.dpo.learning_rate</code>	5.0e-6	不要改回 1e-5 除非你做新实验并写入 summary
<code>data.seed_instructions</code>	环境工程种子路径	自定义种子时改这里并设 AUTOIF_SEED_PATH

改质量阈值后：

```
python code/8_query_score_filter.py
python code/9_sft_data_construction.py
python scripts/prepare_llamafactory_data.py
# 再单独跑 SFT/DPO，或重跑 run_all 的训练段
```

11. 数据文件说明（给报告用）

11.1 种子指令

文件：`sample_data/seed_instruction_环境工程.txt`（40 条）。

写法要求：必须能写成 Python 检查，例如「必须包含四个标题」「恰好三个原因且含证据和复核」「必须以信息不足开头」。不要写「请分析该污染问题」。

`run_all.sh` 使用独立文件 `sample_data/seed_instruction_环境工程.txt`。

重新写出领域种子：

```
python scripts/generate_seed_instructions.py --domain 环境工程 --count 40 \
--output sample_data/seed_instruction_环境工程.txt
```

11.2 查询集与测试集

```
python scripts/generate_env_queries.py
```

会覆盖生成：

- `data/environmental_engineering_queries.jsonl` 训练/合成用；
- `data/environmental_engineering_test.jsonl` 评测用，查询文本与训练集无重叠。

每行字段：`query`、`category`、`difficulty`、`required_knowledge`、`split`，计算题带 `gold`。

场景目标比例：污水处理 40%，监测 20%，大气 15%，固废 10%，噪声 5%，环评与清洁生产 10%。若扩充到 1000+ 条，按同一比例分层，并继续保持训练/测试隔离。

11.3 三层验证模块

- code/env_validators.py: 评测与步骤 7 共用；
- code/env_seed_validators.py: 种子指令的确定性 evaluate()。

12. 逐步调试（不用 run_all 时）

适合课堂讲解或某一步失败后从断点续跑。必须先 setup.sh 成功。

12.1 启动教师模型

```
cd /root/autodl-tmp/AutoIF-LLM
export SUPERVISOR_API_BASE="http://localhost:8000/v1"
export SUPERVISOR_API_KEY="EMPTY"
export SUPERVISOR_MODEL="Qwen/Qwen2.5-7B-Instruct"
export NLI_MODEL_PATH="./models/nli"
export HF_ENDPOINT="https://hf-mirror.com"
export AUTOIF_DOMAIN="环境工程"
export AUTOIF_SEED_PATH="sample_data/seed_instruction_环境工程.txt"
export AUTOIF_QUERY_PATH="data/environmental_engineering_queries.jsonl"

TEACHER_PATH=$(find models/teacher -name "config.json" -path "*/Qwen*" | head -1 | xargs dirname)
echo "教师模型路径: $TEACHER_PATH"

python -m vllm.entrypoints.openai.api_server \
--model "$TEACHER_PATH" \
--served-model-name Qwen/Qwen2.5-7B-Instruct \
--port 8000 --trust-remote-code \
--gpu-memory-utilization 0.5 --max-model-len 4096 &

# 等待就绪（出现模型列表）
sleep 120
curl http://localhost:8000/v1/models
curl http://localhost:8000/health
```

12.2 九步 + DPO

```
python code/1_RFT.py
python code/2_verification_funes_cases_generation.py
python code/3_cross_validation.py
python code/4_eval_func_backtranslator.py
python code/5_eval_func_backtranslator_filter.py
python code/6_concat_sharegpt_query.py
python code/7_query_verification.py
python code/8_query_score_filter.py
python code/9_sft_data_construction.py
python code_dpo/1_dpo_rft_wash.py
python code_dpo/2_dpo_data_query_construct.py
python code_dpo/3_env_hard_negatives.py
python scripts/prepare_llamafactory_data.py
```

某步失败时：看对应 logs/ 或终端 traceback，确认上一步输出文件已生成：

```
ls -lh output/
wc -l output/*.jsonl output/*.txt 2>/dev/null
```

12.3 关 vLLM 再训练

```
pkill -f "vllm.entrypoints" || true
sleep 5
STUDENT_PATH=$(find models/student -name "config.json" -path "*/Qwen*" | head -1 | xargs dirname)
echo "学生模型: $STUDENT_PATH"
```

```

cd LlamaFactory
llamafactory-cli train --stage sft --do_train \
--model_name_or_path "$STUDENT_PATH" \
--dataset autoif_sft --template qwen \
--finetuning_type lora --lora_rank 8 --lora_alpha 16 \
--lora_target q_proj.v_proj \
--output_dir ../models/model_c_sft --overwrite_output_dir \
--per_device_train_batch_size 4 --gradient_accumulation_steps 4 \
--learning_rate 5e-5 --num_train_epochs 3 \
--logging_steps 5 --save_steps 100 --warmup_ratio 0.1 \
--fp16 --cutoff_len 2048 --report_to none

llamafactory-cli train --stage dpo --do_train \
--model_name_or_path "$STUDENT_PATH" \
--adapter_name_or_path ../models/model_c_sft \
--dataset autoif_dpo --template qwen \
--finetuning_type lora --lora_rank 8 --lora_alpha 16 \
--lora_target q_proj.v_proj \
--output_dir ../models/model_c_dpo --overwrite_output_dir \
--per_device_train_batch_size 2 --gradient_accumulation_steps 4 \
--learning_rate 5e-6 --num_train_epochs 2 \
--logging_steps 5 --save_steps 100 --warmup_ratio 0.1 \
--fp16 --cutoff_len 2048 --pref_beta 0.1 --report_to none

llamafactory-cli export \
--model_name_or_path "$STUDENT_PATH" \
--adapter_name_or_path "../models/model_c_dpo" \
--template qwen --finetuning_type lora \
--export_dir ../models/model_merged" \
--export_size 2 --export_legacy_format false
cd ..
python scripts/build_experiment_summary.py

```

40GB 显存 OOM: SFT 的 `--per_device_train_batch_size` 改为 2 或 1, 同时把 `--gradient_accumulation_steps` 加倍, 保持有效 batch 大致不变。也可改 YAML 后用 `run_all.sh`。

13. 下载结果与关机

13.1 小文件（日志 + 数据，推荐先下这个）

```

cd /root/autodl-tmp/AutoIF-LLM
tar -czf results.tar.gz \
logs/ \
output/ \
eval_results/ \
LlamaFactory/data/autoif_sft.json \
LlamaFactory/data/autoif_dpo.json \
data/environmental_engineering_queries.jsonl \
data/environmental_engineering_test.jsonl
du -sh results.tar.gz

```

JupyterLab 里对 `results.tar.gz` 右键 Download。

13.2 完整合并模型（约 3GB）

在本地：

```
scp -r -P 端口号 root@connect.xxx.seetacloud.com:/root/autodl-tmp/AutoIF-LLM/models/model_merged/ ./model_merged/
```

13.3 关机

回到 AutoDL 网页 → 容器实例 → 关机。

- 关机：数据盘还在，下次开机可继续；

- 释放实例：磁盘也会删，先确认结果已下载。

不关机就会一直计费。

14. 常见问题（按出现顺序）

Q1 磁盘不足 / No space left

项目不在 `/root/autodl-tmp`。移过去再 `setup.sh`。

Q2 模型下载超时

重跑 `setup.sh`，已下载的会跳过。

Q3 vLLM 启动超时

```
nvidia-smi
find models/teacher -name "config.json" | head
# 按 12.1 节手动启动，看完整报错
```

显存不够时把 `--gpu-memory-utilization` 降到 0.4，或换 80GB 卡。

Q4 404 模型不存在

```
curl http://localhost:8000/v1/models
export SUPERVISOR_MODEL="这里必须等于 served-model-name"
```

Q5 SFT 不到 100 条或全是空 output

空 output：检查是否误读 `gpt-answer`，应使用 `IF_sft_data.json`。条数太少：把质量分阈值降到 5~6，重跑 step 8-9。

Q6 训练 OOM

减小 batch size，增大 gradient accumulation。先 `kill` 掉 vLLM。

Q7 量化失败

忽略。最终模型是 `model_merged`。

Q8 关浏览器后任务没了

没用 `nohup`。以后一律 `nohup ... &`。

Q9 想跑污水处理 / 环境监测

```
bash run_all.sh --domain 污水处理
bash run_all.sh --domain 环境监测
```

二者与环境工程共用种子模板。查询集仍建议显式传入 `--query-file`。

Q10 如何证明有效

用测试集跑 `eval_env_engineering.py`，报计算正确率、单位正确率、虚构标准率，对比基座和 SFT+DPO。不要只贴 Loss 曲线。

Q11 查询仍像闲聊

确认 `AUTOIF_QUERY_PATH` 指向 `data/environmental_engineering_queries.jsonl`，且文件存在。步骤 6 日志里应打印「从领域查询集加载 xxx 条」。

Q12 DPO 对很少

需要同一问题下同时有高分和 0 分回答。可看 `logs/dpo2.log`。硬负样本在 `dpo3` 会追加一批。

Q13 种子被覆盖

当前脚本使用独立文件。若你用的是旧版 `run_all.sh` 才会备份/覆盖。请确认项目根目录的 `run_all.sh` 含 `AUTOIF_SEED_PATH` 且注释写了「未覆盖原始种子」。

Q14 本地 Windows 能跑全流程吗

不能按 `AutoDL` 方式跑教师模型+训练，除非你自己有 `GPU` 和 `Linux` 环境。Windows 上可以跑：`tests/test_env_validators.py`、`scripts/demo_env_autoif.py`、查询生成、PDF/文档。全流程请上 `AutoDL`。

15. 命令速查（复制区）

安装与全流程：

```
cd /root/autodl-tmp/AutoIF-LLM
bash setup.sh
nohup bash run_all.sh --domain 环境工程 \
--query-file data/environmental_engineering_queries.jsonl \
--test-file data/environmental_engineering_test.jsonl \
> run.log 2>&1 &
tail -f run.log
```

只看进度：

```
tail -f /root/autodl-tmp/AutoIF-LLM/run.log
```

评测与汇总：

```
python scripts/build_experiment_summary.py
python scripts/eval_env_engineering.py --model-path ./models/model_merged --tag sft_dpo
python tests/test_env_validators.py
python scripts/demo_env_autoif.py
```

单题手工推理（合并模型已存在时）：

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch
model_path = './models/model_merged'
tokenizer = AutoTokenizer.from_pretrained(model_path, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(
model_path, trust_remote_code=True, device_map='auto', torch_dtype=torch.float16)
prompt = '某污水厂设计水量为10000 m³/d，进水COD为300 mg/L，出水COD为50 mg/L，请计算COD去除率和每日去除负荷。'
messages = [{'role': 'user', 'content': prompt}]
text = tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
inputs = tokenizer(text, return_tensors='pt').to(model.device)
outputs = model.generate(**inputs, max_new_tokens=300, temperature=0.3, do_sample=True)
print(tokenizer.decode(outputs[0][inputs['input_ids'].shape[1]:], skip_special_tokens=True))
```

16. 介绍项目时建议怎么说（口头）

一句话：这不是把通用模型换成环保词，而是给 `AutoIF` 接上环境工程查询集、可执行计算验证器和专业偏好数据，提升污水处理计算、工艺比较、异常诊断和监测报告中的结构化指令遵循。

三个必答题：

1. 为什么选污水处理？计算可验证、流程和报告字段固定，契合 `AutoIF`。
2. 和套 `Prompt` 有何不同？`Prompt` 管不住算错和编造标准；这里用程序抽数字对 `gold`，`DPO` 再拉开错误单位和虚构限值。
3. 模型懂环境工程了吗？没有。学的是按约束把计算和报告写完整；专业事实仍要标准文本或人工抽检。

17. 文档与数字纪律

- README、本操作指南、讲义 PDF，数字只抄 `output/experiment_summary.json`。

- SFT 学习率只写 5e-5，DPO 只写 5e-6。
- 操作步骤以 AutoIF环境工程操作指南.pdf 为准。

附录 A 目录结构（操作时会用到的部分）

```
AutoIF-LLM/
setup.sh / run_all.sh / README.md
configs/pipeline_config.yaml
sample_data/seed_instruction_环境工程.txt
data/environmental_engineering_queries.jsonl
data/environmental_engineering_test.jsonl
code/1_RFT.py ... 9_sft_data_construction.py
code/env_validators.py config_loader.py
code_dpo/1 ... 2 ... 3_env_hard_negatives.py
scripts/prepare_llamafactory_data.py
scripts/eval_env_engineering.py
scripts/build_experiment_summary.py
scripts/demo_env_autoif.py
tests/test_env_validators.py
docs/AutoIF环境工程操作指南.md
AutoIF环境工程操作指南.pdf
output/ 运行后生成
logs/ 运行后生成
models/ setup 后生成
```

底层仍是 AutoIF 九步 + LlamaFactory
LoRA/DPO。改过的是领域数据、验证器和训练数据组装，不是从零写训练框架。

附录 B 检查清单（答辩前自己过一遍）

- [] 实例在数据盘 /root/autodl-tmp/AutoIF-LLM
- [] 镜像 PyTorch 2.4.0 + CUDA 12.1
- [] setup.sh 看到环境配置完成，CUDA 为 True
- [] 使用 --domain 环境工程 且传入查询集、测试集
- [] 用了 nohup，日志里有「使用独立种子文件」
- [] IF_sft_data.json 条数非 0 且 output 非空
- [] dpo_pairs.jsonl 能看到「环境问题」和「格式要求」
- [] 已生成 experiment_summary.json 并只引用它
- [] 量化失败没有写成项目失败
- [] 已做至少基座 vs 合并模型的计算题抽检
- [] AutoDL 实例已关机